

Code Verification via Method of Manufactured Solutions

By Marvyn Bailly

0.1 Recreating Results

Let's begin by trying to recreate the Code Verification through the Method of Manufactured Solutions (MMS). The paper is "Code Verification by the Method of Manufactured Solutions" by Patrick Roache.

0.1.1 Simple Example

Let's begin by considering the simple advection equation

$$L(u) \equiv u_t + au_x = 0, \quad (1)$$

subject to the initial condition $u(x, 0) = f(x)$. Using the Method of Characteristics, we know that the true solution to the PDE remains constant along its characteristics, giving rise to the solution $u(x, t) = f(x - at)$ (modulo boundary conditions). Now, we would like to manufacture a solution to $U(x, t)$. While $U(x, t)$ may not be a solution to $L(u)$, it is the solution to $L(u) = Q(x, t)$ where $Q(x, t)$ is a source term that we must determine. That is,

$$Q(x, t) = L(U(x, t)) = \partial_t U(x, t) + a\partial_x U(x, t). \quad (2)$$

Now the manufactured solution $U(x, t)$ is a solution to $L(u) = Q(x, t)$ or

$$u_t + au_x = Q = \partial_t U(x, t) + a\partial_x U(x, t), \quad (3)$$

where $U(x, 0) = u(x, 0) = f(x)$. This means that the manufactured solution must be compatible with the prescribed initial condition. We must also make sure that $U(x, t)$ is consistent with the boundary conditions. For the sake of simplicity, let's prescribe periodic boundary conditions and worry about different types of boundary conditions later.

Accuracy

Let's test the method using Lax-Friedrichs (LxF) on 1. Recall that LxF is given by

$$v_\nu^{n+1} = \frac{1}{2}(v_{\nu+1}^n - v_{\nu-1}^n) - \frac{ka}{h2}(v_{\nu+1}^n + v_{\nu-1}^n).$$

Thus LxF applied to 3 yields

$$v_\nu^{n+1} = \frac{1}{2}(v_{\nu+1}^n - v_{\nu-1}^n) - \frac{ka}{h2}(v_{\nu+1}^n + v_{\nu-1}^n) + kQ(x, t),$$

where $Q(x, t)$ is from 2. Let's quickly verify that the accuracy LxW applied 1 (1st order in time and space) is the same as 3. By Taylor expanding the RHS in space we get

$$\begin{aligned} v_\nu^{n+1} &= \frac{1}{2}(v_\nu^n + h^2(v_\nu^n)_{xx} + \mathcal{O}(h^3)) - \frac{ka}{h^2} \left(2h(v_\nu^n)_x + 2\frac{h^3}{3!}(v_\nu^n)_{xxx} + \mathcal{O}(h^5) \right) + kQ(x, t) \\ &= v_\nu^n + \frac{h^2}{2}(v_\nu^n)_{xx} - ka((v_\nu^n)_x + h^2(v_\nu^n)_{xxx}) + kQ(x, t) + \mathcal{O}(h^3). \end{aligned}$$

Now temporally expanding the LHS yields

$$v_\nu^{n+1} = v_\nu^n + k(v_\nu^n)_t + \mathcal{O}(k^2),$$

and using 3 we can swap temporal derivatives for spatial yielding

$$v_\nu^{n+1} = v_\nu^n + k(Q - a(v_\nu^n)_x) + k^2(v_\nu^n)_{tt} + \mathcal{O}(k^3),$$

Now subtracting the LHS from the RHS and dropping higher order terms yields

$$\begin{aligned} \text{LTE} &= \frac{h^2}{2}(v_\nu^n)_{xx} + kh^2(v_\nu^n)_{xxx} + k^2(v_\nu^n)_{tt} \\ &= \mathcal{O}(k^2 + h^2). \end{aligned}$$

which aligns with LxF.

0.1.2 Numerical Test

Let's use LxF to solve 1 with $a = 4$ subject to $u(x, 0) = f(x)$ where $f(x) = \sin(x)$ with 2π -periodic boundary conditions.

Let's do it in Julia! Okay the basic solver and example have been set up in `example_working.jl`.

Next, let's create a Julia and use Symbolics.jl to handle the symbolic calculation of the modified solution. Let's create the modified solution

Note: Looks like we can write C code using Julia! https://symbolics.juliasymbolics.org/dev/tutorials/converting_to_C/

$$U(x, t) = \cos(t).$$

But we require $U(x, 0) = u(x, 0) = f(x)$ so for us,

$$U(x, 0) = \cos(0) \neq \sin(x),$$

so let's modify $U(x, t)$ to be

$$U(x, t) = 1 - \cos(t) + \sin(x) \implies U(x, 0) = 1 - \cos(0) + \sin(x) = \sin(x) = f(x) = u(x, 0),$$

as desired. Now let's compute $Q(x, t)$ by hand first:

$$Q(x, t) \equiv L(U(x, t))$$

$$\begin{aligned}
&\equiv \partial_t U(x, t) + a \partial_x U(x, t) \\
&= \frac{\partial}{\partial t} (1 - \cos(t) + \sin(x)) + a \frac{\partial}{\partial x} (1 - \cos(t) + \sin(x)) \\
&= \sin(t) + a \cos(x).
\end{aligned}$$

Now we implement the same behavior in Julia using Symbolics.jl Now let's implement this into Julia:

```

1  @info("Beginning symbolic fun")
2
3  # Define variables
4  @variables x,t, u(x,t)
5
6  # Define differentials
7  Dx = Differential(x)
8  Dt = Differential(t)
9
10 # Define the PDE
11 a = 4
12 pde = Dt(u) + a*Dx(u)
13
14 # Define manufactured solution
15 manu = exp(x*t)*cos(t)*sin(x)
16
17 # The manufactured solution is the true solution to the modified
    PDE
18 true_sol = build_function(manu,[x,t], expression=Val{false})
19
20 # Substitute manufactured solution into PDE
21 Q = substitute(pde, u => manu)
22 # simplified_Q = expand_derivatives(Q)
23 # println(simplified_Q)
24
25 forcing_term = expand_derivatives(Q)
26
27 # turn it into a string / Make into whatever we need s.t. we can
    add it to the lang
28 forcing_term_str = string(forcing_term)
29 if debug
30     @info("The force terming is given by: Q(x,t)=$forcing_term_str
        ")
31 end

```

Now we want to automatically add the forcing term to the coders script

```

1 @info("Adding the forcing term")
2
3 # now let's read in the script
4 script_contents = read(file_location , String)
5
6 # and search for '# INSERT_FORCING_TERM_HERE
7 modified_script_contents = replace(script_contents , "#
    INSERT_FORCING_TERM_HERE" => "$forcing_term_str")
8
9 # AND SEARCH FOR '# INSERT_STOP_TIME_HERE
10 modified_script_contents = replace(modified_script_contents , "#
    INSERT_STOP_TIME_HERE" => "$t_final")
11
12 # write the file with the forcing term
13 write("modified_script_contents.jl", modified_script_contents)

```

which searches the user's code for comments `# INSERT_FORCING_TERM_HERE` and replaces it with the symbolic term.

Now that we have the correct forcing term in the script, we want to run the code for two different grid values. We are aiming to verify the order-of-accuracy of the code. So if E_1 and E_2 are the global errors corresponding to a grid size of h and h/r respectively, where r is the refinement ratio. If the method is expected to be p accurate, then

$$\frac{E_1}{E_2} \leq \left(\frac{C_1 h^p}{C_2 \frac{h^p}{r^p}} \right) \approx r^p,$$

where C_1 and C_2 is some constant. Then we have that

$$p \approx \frac{\log \left(\frac{E_1}{E_2} \right)}{\log r}.$$